



**DEPARTAMENTO
DE COMPUTACION**

Facultad de Ciencias Exactas y Naturales - UBA

Trabajo práctico

El subtítulo adecuado

September 1, 2026

Algoritmos y Estructuras de Datos

Java con Mates

Integrante	LU	Correo electrónico
Ayala, Ignacio	196/25	ayalanacho11@gmail.com
Della Bonzana, Lara	563/25	dellabonzanalara@gmail.com
Morales, Santiago	481/25	santumorales11@gmail.com
Vannelli, Tiziana	945/25	tizivannelli@gmail.com



Facultad de Ciencias Exactas y Naturales

Universidad de Buenos Aires

Ciudad Universitaria - (Pabellón I/Planta Baja)

Intendente Güiraldes 2610 - C1428EGA

Ciudad Autónoma de Buenos Aires - Rep. Argentina

Tel/Fax: (+54 +11) 4576-3300

<http://www.exactas.uba.ar>

1 Macros para escribir en LaTeX

Para facilitar la escritura del trabajo práctico en LaTeX, les proveemos de varias macros que les permitirán escribir especificaciones en lógica de primer orden. Por ejemplo, podrán hacer uso de comandos que definen los conectores lógicos: $\wedge, \vee, \rightarrow$ y también los operadores de la lógica trivaluada $\wedge_L, \vee_L, \rightarrow_L$.

Además, disponen de comandos para cuantificadores: $(\forall x : \mathbb{Z}) (x \geq 0 \vee x < 0)$, $(\exists x : \mathbb{Z}) (x \geq 0)$. A estos comandos se les puede agregar un **Largo** al final para que ocupen múltiples renglones.

2 Ejemplos de uso de las macros

```
TAD EjemploDeTAD {
  obs secuenciaDeCaracteres : seq⟨char⟩
  obs conjuntoDeTuplas : conj⟨⟨nombre : seq⟨char⟩, apellido : seq⟨char⟩⟩⟩

  proc crearTAD(in caracteres : seq⟨char⟩, in secuenciaDeNombres : seq⟨⟨nombre : seq⟨char⟩, apellido :
  seq⟨char⟩⟩⟩) : EjemploDeTAD {
    requiere { predicadoUnilinea(caracteres) }
    asegura { res.secuenciaDeCaracteres = caracteres }
    asegura {
      |res.conjuntoDeTuplas| = |secuenciaDeNombres|  $\wedge$ 
      predicadoMultilinea(secuenciaDeNombres, res.conjuntoDeTuplas)
    }
  }
  pred predicadoMultilinea(s : seq⟨char⟩, conjunto : conj⟨⟨seq⟨char⟩, seq⟨char⟩⟩⟩) {
     $(\forall i : \mathbb{Z}) (0 \leq i < |s| \rightarrow_L s[i] \in c)$ 
  }
  pred predicadoUnilínea(s : seq⟨char⟩) { |s| > 0 }
  aux unAuxiliar(k :  $\mathbb{Z}$ , l :  $\mathbb{Z}$ ) :  $\mathbb{Z} = |k - l|$ 
}
```

Usuario ES $\langle id : \mathbb{Z}, millasTotales : \mathbb{Z}, millasDisponibles : \mathbb{Z}, historial : seq\langle, \rangle factor : \mathbb{Z} \rangle$
 Promocion ES $\langle nombre : char, factor : \mathbb{Z}, estado : bool \rangle$
 Categoria ES char

TAD AirMiles {

```

    obs usuarios : conj⟨Usuario⟩
    obs promociones : conj⟨Promocion⟩
    proc crearSistema() : AirMiles {
      requiere { True }
      asegura { (∃p : Promocion) (p ∈ res.promociones ∧ p.nombre = ' NoPromo' ∧
        p.factor = 1 ∧ p.estado = True) }
      asegura { res.usuarios = ∅ }
      asegura { #res.promociones = 1 }
    }
    proc registrarUsuario(inout a : AirMiles, in id : ℤ) {
      requiere { !esUsuarioRegistrado(id, a) ∧ a = A0 }
      asegura { #a.usuarios = #A0.usuarios + 1 }
      asegura { (∀u : Usuario) (u ∈ A0.usuarios →L u ∈ a.usuarios) }
      asegura { (∃u : Usuario) (u ∉ A0.usuarios ∧ u ∈ a.usuarios ∧ u.id = id ∧ u.millasDisponibles =
        0 ∧ u.millasTotales = 0 ∧ u.historial = [] ∧ u.categoria = ' Bronce') }
      asegura { a.promociones = A0.promociones }
    }
    proc registrarUsuarioConReferidor(in idNuevoUsuario : ℤ, in referidor : Usuario, inout a :
    AirMiles) {
      requiere { referidor ∈ a.usuarios ∧ a = A0 ∧ !esUsuarioRegistrado(idNuevoUsuario, a) }
      asegura { (∃u : Usuario) (u ∈ a.usuarios ∧ u.id = idNuevoUsuario ∧ u.millasTotales =
        500 ∧ u.millasDisponibles = 500 ∧ u.historial = [] ∧ u.categoria = Bronce) }
      asegura { (∀u : Usuario) (u ∈ A0.usuarios ∧ u ≠ referidor →L u ∈ a.usuarios) }
      asegura { #a.usuarios = #A0.usuarios + 1 }
      asegura { (∃u : Usuario) (u.id = referidor.id ∧ u.millasDisponibles = referidor.millasDisponibles +
        500 ∧ u.millasTotales = referidor.millasTotales + 500 ∧ u.historial = referidor.historial +
        + ∧ u.categoria = queCategoriaEs(u.millasTotales, a)) }
      asegura { a.promociones = A0.promociones }
    }
    proc acumularMillas(in usuario : Usuario, in distancia : ℤ, in nombrePromo : char, inout a :
    AirMiles) {
      requiere { usuario ∈ a.usuarios ∧ a = A0 ∧ distancia ≥ 0 ∧ esPromoActiva(nombrePromo, a) }
    }
    proc canjearMillas(in u : Usuario, in cantMillas : ℤ, in premio : T, inout a : AirMiles) {
      requiere { cantMillas > 0 ∧ u ∈ a.usuarios ∧ a = A0 ∧ u.millasDisponibles ≥
        cantMillas }
      asegura { (∀u : Usuario) (u ∈ A0.usuarios ∧ u ≠ ref →L u ∈ a.usuarios) }
      asegura { #A0.usuarios = #a.usuarios }
      asegura { (∃u0 : Usuario) (u0.id = u.id ∧ u0.millasDisponibles = u.millasDisponibles -
        cantMillas ∧ u.millasTotales = u0.millasTotales ∧ u0.historial = u.historial + + [] ∧
        u0.rango = u.rango) }
      asegura { A0.promociones = a.promociones }
  
```

```

}
proc transferirMillas(in cantMillas :  $\mathbb{Z}$ , in usuarioOrigen : Usuario, in usuarioDestino :
Usuario, inout a : AirMiles) {
  requiere { usuarioDestino  $\in$  a.usuarios  $\wedge$  usuarioOrigen  $\in$  a.usuarios  $\wedge$  cantMillas  $\geq$ 
0  $\wedge$  usuarioOrigen.millasDisponibles  $\geq$  cantMillas  $\wedge$  usuarioDestino  $\neq$  usuarioOrigen  $\wedge$ 
a = A0 }
  asegura { ( $\forall u$  : Usuario) ( $u \in$  A0.usuarios  $\wedge u \neq$  usuarioOrigen  $\wedge u \neq$  usuarioDestino  $\rightarrow$ 
u  $\in$  a.usuarios) }
  asegura { #A0.usuarios = #a.usuarios }
  asegura { ( $\exists u_0$  : Usuario) () }
}
proc consultarSaldo(in u : Usuario, in a : AirMiles) :  $\mathbb{Z}$  {
  requiere { u  $\in$  a.usuarios }
  asegura { res = u.millasDisponibles }
}
proc consultarCategoria(in u : Usuario, in a : AirMiles) : Categoria {
  requiere { u  $\in$  a.usuarios }
  asegura { res = u.categoria }
}
proc rankingMillas(tipos) : AirMiles {
  requiere { True }
  asegura { algo }
}
proc resumenTransacciones(tipos) : AirMiles {
  requiere { True }
  asegura { algo }
}
proc iniciarPromocion(in nombre : char, in factor :  $\mathbb{Z}$ , inout a : AirMiles) {
  requiere { f  $\geq$  1  $\wedge$  a = A0 }
  asegura { ( $\exists p$  : Promocion) ( $p \in$  a.promociones  $\wedge p.nombre =$  nombre  $\wedge p.factor =$ 
factor  $\wedge p.estado =$  True) }
  asegura { ( $\forall p$  : Promocion) ( $p \in$  A0.promociones  $\wedge p.nombre \neq$  nombre  $\rightarrow p \in$ 
a.promociones) }
  asegura { a.usuarios = A0.usuarios }
}
proc iniciarPromocionConTope(tipos) : AirMiles {
  requiere { True }
  asegura { algo }
}
proc finalizarPromocion(in nombre : char, inout a : AirMiles) {
  requiere { ( $\exists p$  : Promocion) ( $p \in$  a  $\wedge p.nombre =$  nombre  $\wedge$  PromoActiva(nombre, a))a =
A0 }
  asegura { algo }
}
proc elQueTieneMasMillas(in a : AirMiles) :  $\mathbb{Z}$  {
  requiere { True }
  asegura { ( $\exists u$  : Usuario) ( $u \in$  a.usuarios  $\wedge u.id =$  res  $\wedge$  tieneMaximasMillas(u, a))
}
}

```

```

}
proc elMasCanjeador(tipos) : AirMiles {
  requiere { True }
  asegura { algo }
}
proc paresTransferidoresExclusivos(tipos) : AirMiles {
  requiere { True }
  asegura { algo }
}
aux queCategoriaEs(millas :  $\mathbb{Z}$ , a : AirMiles) : Categoria = IfThenElse(millas  $\leq$  10000, Bronce,
IfThenElse(millas  $\leq$  50000, Plata, IfThenElse{millas  $\leq$  1000000, Oro, Platino}))

aux factorDeCategoria(categoria : Categoria) :  $\mathbb{Z}$  = IfThenElse(categoria = Bronce, 1, IfThenElse(categoria = Plata, 1.5, IfThenElse(categoria = Oro, 2, 0)))
aux calculoMillas(distancia :  $\mathbb{Z}$ , factorCategoria :  $\mathbb{Z}$ , factorPromocion :  $\mathbb{Z}$ ) :  $\mathbb{Z}$  =
distancia * factorCategoria * factorPromocion
pred tieneMaximasMillas(u : Usuario, a : AirMiles) { ( $\forall u_0$  : Usuario) ( $u_0 \in a.usuarios \wedge$ 
 $u_0 \neq u \rightarrow u.millasTotales \geq u_0.millasTotales$ ) }
pred esUsuarioRegistrado(id :  $\mathbb{Z}$ , a : AirMiles) { ( $\exists u_0$  : Usuario) ( $u_0 \in a.usuarios \wedge_L$ 
 $u_0.id = id$ ) }
pred esPromoActiva(nombre : char, a : AirMiles) { ( $(\exists p$  : Promocion) ( $p \in a.promociones \wedge_L$ 
 $p.nombre = nombre \wedge p.estado = True$ )) }
}

```